Loan Default Prediction using Random Forest

Prasad Bhalerao (RBT24CB026) · Faculty: Prof. Neeraj Sathawane · Subject: Machine Learning

JSPM's
Rajarshi Shahu College of Engineering, Tathawade, Pune
(An Autonomous Institute affiliated to Savitribai Phule Pune University)
Department of Computer Science and Business Systems

Academic Year: 2026-27

Project Presentation

Project Title:

Loan Default Prediction using Random Forest

Students Name and PRN no: Prasad Bhalerao (RBT24CB026)

Faculty: Prof. Neeraj Sathawane

Subject: Machine Learning

1 · Title

Dataset and Problem

Give Me Some Credit (Kaggle): who is in the data, what we predict, what had to be fixed

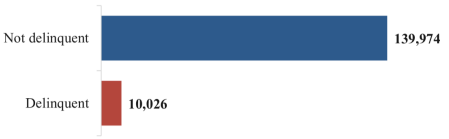
150,000	10	6.68%
labelled borrowers	numeric predictors	delinquent borrowers

Ten predictors, in three groups

Payment history	Credit usage	Borrower profile
30–59 days late	Revolving utilisation	Age
60–89 days late	Debt ratio	Monthly income
90+ days late	Open credit lines	Dependents
	Real-estate loans	

Target: SeriousDlqin2yrs
1 = 90+ days past due within two years 0 = otherwise

Class balance: the delinquent class is rare



Always answering "no" scores 93.3% accuracy and catches no one.

Data issues found, and what was done

Issue	Affected	Handling
Monthly income missing	19.82%	Median + flag
Dependents missing	2.62%	Median + flag
Late counts coded 96 / 98	269 rows	Flag: 54.65% positive
Age equal to 0	1 row	Treated as missing
Outliers, duplicate rows	kept	Trees are rank-based

2 · Dataset and Problem

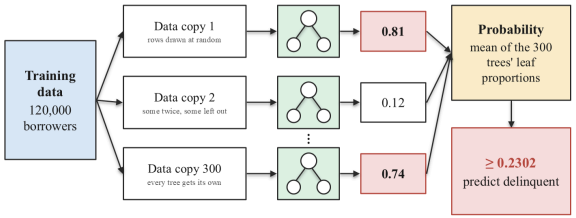
Methodology and Algorithm

A leakage-free pipeline around a Random Forest classifier

1 Load	2 Clean + impute	3 Split	4 Compare	5 Set cut-off	6 Test once
150,000 labelled rows, 10 predictors, row index dropped	96/98 codes and age < 18 become missing. Median fill plus missing flags	Stratified 60 / 20 / 20: 90,000 train, 30,000 validation, 30,000 test	Five forests fitted on train, ranked by validation PR-AUC	Threshold 0.2302 chosen on validation by maximum F1	Refit on train + validation, then score the untouched test set

Every statistic is fitted on training data only. The test set is touched exactly once.

How the forest produces a prediction



Design choices, all decided on validation data

01 **300 trees, minimum leaf size 20**
Limits overfitting: validation PR-AUC 0.3820 against 0.3481 for unrestricted trees

02 **Class weighting tested, not used**
PR-AUC stayed near 0.3776: it only slides along the same precision–recall curve

03 **Missing-value flags added**
Best validation PR-AUC (0.3863). The 96/98 rows are 54.65% positive

04 **Cut-off 0.2302, not 0.50**
Validation F1 0.4396 held up on the test set (0.4482)

3 · Methodology and Algorithm

Results and Evaluation

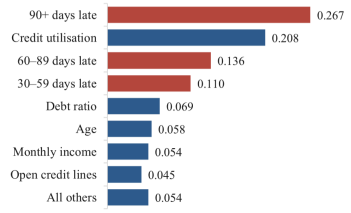
Held-out test set of 30,000 borrowers, scored exactly once

0.8654	0.4014	0.4868	0.4153	0.4482
ROC-AUC threshold-free	**PR-AUC** chance level 0.067	**Recall** caught 976 of 2,005	**Precision** at cut-off 0.2302	**F1-score** at cut-off 0.2302

Confusion matrix

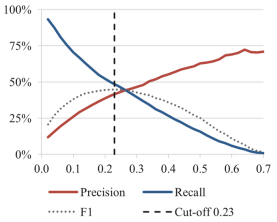
	Predicted not delinquent	Predicted delinquent
Actual not delinquent	26,621 true negatives	1,374 false alarms
Actual delinquent	1,029 missed	976 caught

What the forest relies on



Payment history (red) holds 51% of importance. Not causation.

Moving the cut-off



Lower cut-off: more caught, more false alarms.

Reading it: accuracy (92.0%) is below the 93.3% of always answering "no", so PR-AUC and recall carry the result. Five-fold cross-validation agrees: ROC-AUC 0.8632 ± 0.0053. **Limits:** 2011 US data, uncalibrated probabilities, no fairness test. An educational prototype.

4 · Results and Evaluation

Technical Sheet — algorithm, metrics and results

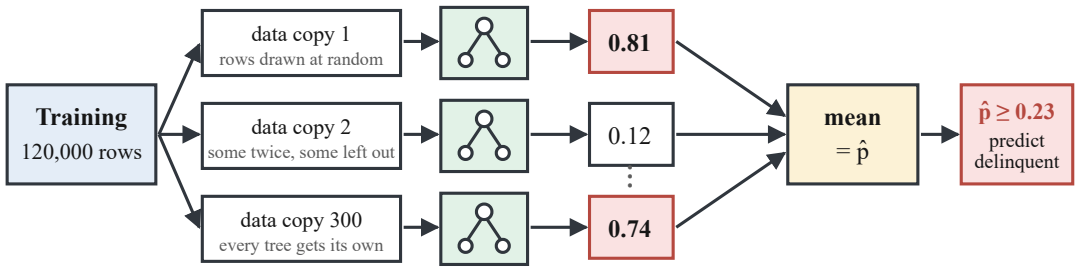
Loan Default Prediction using Random Forest · Prasad Bhalerao (RBT24CB026)

1 · Problem

Supervised binary classification. From 10 numeric borrower attributes, estimate P(serious delinquency within 2 years), then threshold it to a class. Not regression (the output is a category) and not clustering (the groups come from labels).

150,000	10	6.68%
borrowers	features	positive

2 · Random Forest — how a prediction is made



- **Make 300 copies of the data.** Each copy is 120,000 rows drawn at random *with replacement* — a row can appear twice or not at all. Every tree therefore learns from slightly different data. (The statistics name for this resampling is a *bootstrap*; building one tree per copy is *bagging*.)
- **Grow one tree per copy.** At each node, look at a *random handful of the 10 features* — not all of them — and take the split that separates the classes best. Random features keep the trees from being near-copies of each other, which is what makes averaging them worthwhile.
- **Stop** when a leaf holds fewer than 20 borrowers.

Choosing a split — Gini impurity

$$G = 1 - \Sigma\, p_k^2$$

0 = pure node. Take the split with the largest weighted drop in G:

$$\Delta G = G_{parent} - (n_L/n)G_L - (n_R/n)G_R$$

3 · The prediction, step by step

1 · Each tree answers

$$\frac{n^+ \text{ in leaf}}{n \text{ in leaf}} = p_t(x)$$

Borrower x falls into one leaf. That tree reports the share of that leaf's training borrowers who defaulted.

2 · The forest averages

$$\hat{p}(x) = \frac{1}{T}\Sigma_{t=1}^{T} p_t(x)$$

T = 300. The **mean of the 300 proportions** — not a majority vote of 300 yes/no answers.

3 · Compare to the cut-off

$$\hat{y} = 1 \text{ if } \hat{p}(x) \geq \tau$$

τ = **0.2302**, fixed on validation data — not the usual 0.50. Below τ → predict "no delinquency".

Worked example (the diagram above): the trees return 0.81, 0.12, … , 0.74. Their mean is p̂ = 0.43. Since 0.43 ≥ 0.2302, the borrower is flagged. The reported figure is that 0.43, a *ranking score* — not a claim that 43 in 100 such borrowers default.

4 · Evaluation metrics — from TP / FP / TN / FN

Accuracy
$\frac{TP + TN}{N}$
Misleading when one class is rare

Precision
$\frac{TP}{TP + FP}$
Of those flagged, how many were right

Recall (TPR)
$\frac{TP}{TP + FN}$
Of the truly delinquent, how many found

F1
$2 \cdot \frac{P \cdot R}{P + R}$
Harmonic mean — the weaker one dominates

FPR
$\frac{FP}{FP + TN}$
Divides by 27,995 negatives → flattering

ROC-AUC
$P(s^+ > s^-)$
Area under TPR–FPR. Chance = 0.5

PR-AUC
$\int P\, dR$
Baseline = positive rate = 0.0668

Why PR-AUC here
FPR's huge denominator hides false alarms. Precision divides only by what was flagged, so it stays honest.

5 · Results — held-out test set (30,000 borrowers, scored once)

	pred. 0	pred. 1
actual 0	26,621 TN	1,374 FP · false alarm
actual 1	1,029 FN · missed	976 TP · caught

0.8654	0.4014	0.4868	0.4153	0.4482
ROC-AUC	PR-AUC	Recall	Precision	F1

Accuracy 0.9199 — below the 93.3% of always answering "no". That is the point. Caught 976 of 2,005 delinquents. 5-fold CV ROC-AUC 0.8632 ± 0.0053; gap train−test +0.0675 ROC, +0.1218 PR.

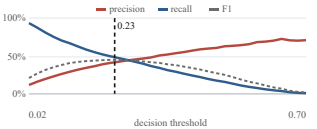
Feature importance

90+ days late	0.267
Credit utilisation	0.208
60–89 days late	0.136
30–59 days late	0.110
Debt ratio	0.069
Age	0.058

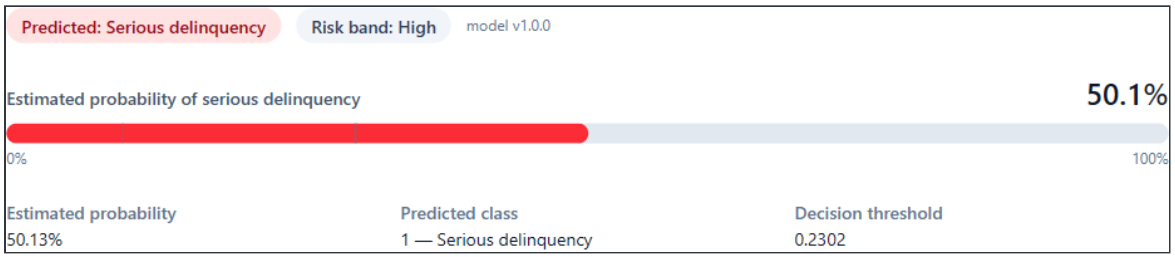
Payment history (red) = 51%. Describes splits, not causation.

6 · Class imbalance and the decision threshold

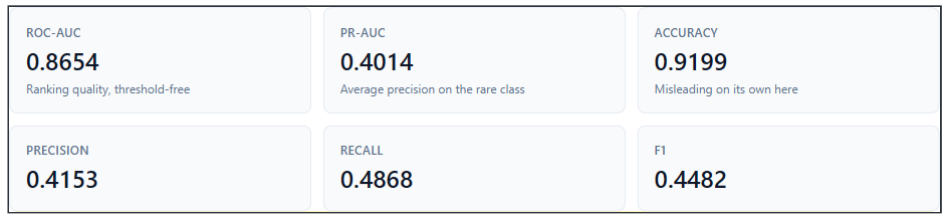
- Only **6.68%** positive. Three remedies: resampling, class weighting, moving the threshold.
- **Class weighting tested, rejected** — recall 0.1556 → 0.7197 but PR-AUC 0.3820 → 0.3776: it slides *along* the curve, it does not lift it.
- **Threshold chosen on validation** by maximising F1 → **0.2302**, not 0.50. Validation F1 0.4396 → test 0.4482.
- **Protocol.** Stratified 60/20/20; statistics fitted on train only; 5 configurations compared on validation; test touched once.



7 · The same arithmetic, running live · delinquency-risk.vercel.app



Panel 3, executed. 86% utilisation, four late episodes → the 300 trees average to **p̂ = 0.501**; 0.501 ≥ 0.2302, so class 1.



Panel 5, served. Read from the model file at start-up — nothing here is typed by hand.

Limitations. US data from ~2011 with no date column, so no out-of-time validation — present-day performance unknown. Probabilities are **uncalibrated**: 0.30 is a ranking score, not "30 in 100 default". No protected attributes, so fairness cannot be tested — absence of measured bias is not evidence of fairness. Maximising F1 weights a miss and a false alarm equally; lending economics do not. **Educational prototype — not for real credit decisions.**